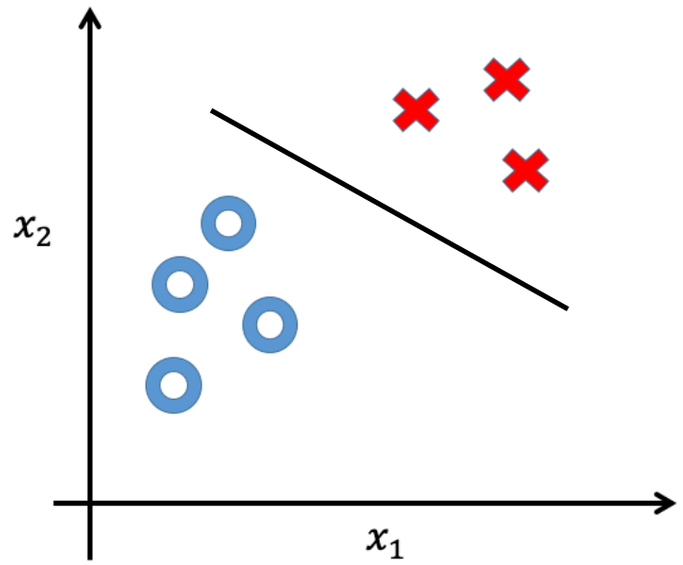
Clustering

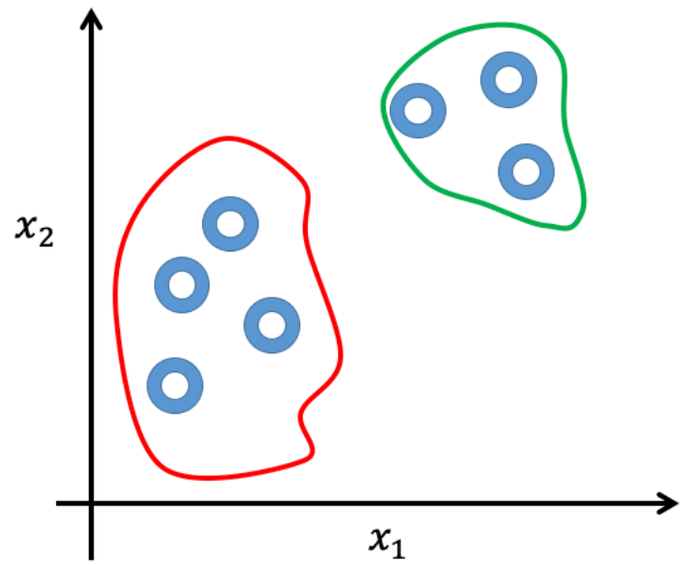
Machine Learning
Hoda Eldardiry

Supervised vs. Unsupervised

- **Supervised Learning**
  - Data + labels
  - Learn decision function
  - Predict labels for new data



- **Unsupervised Learning**
  - Data only
  - No labels
  - Discover structure



What Problems Need Unsupervised Learning?

- Clustering
- Anomaly detection
- Change detection
- Dimensionality reduction
- *Unsupervised learning is about discovering structure without guidance*

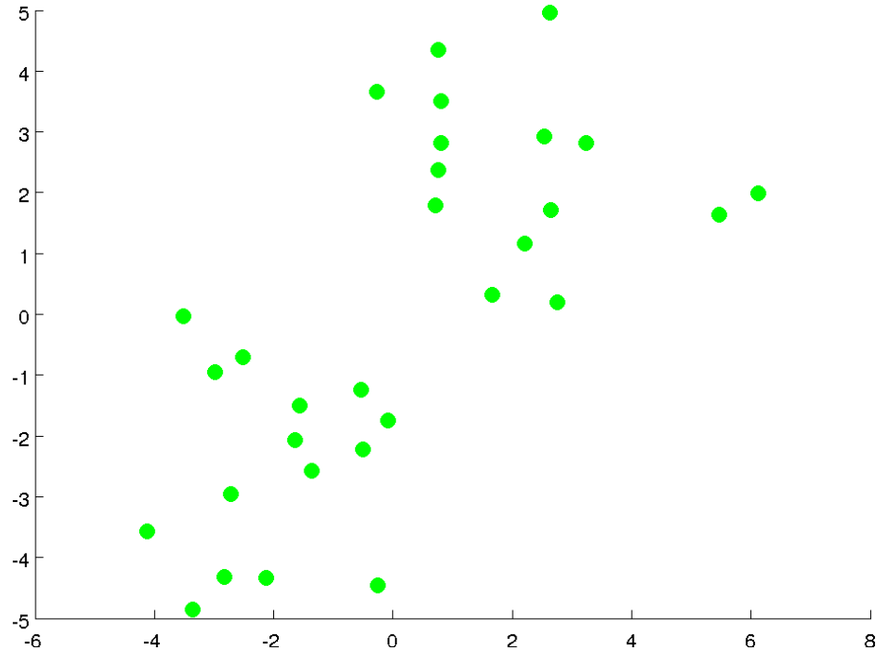
Your Final Project?

- Which teams are considering unsupervised learning?

- Which method?

What is Clustering?

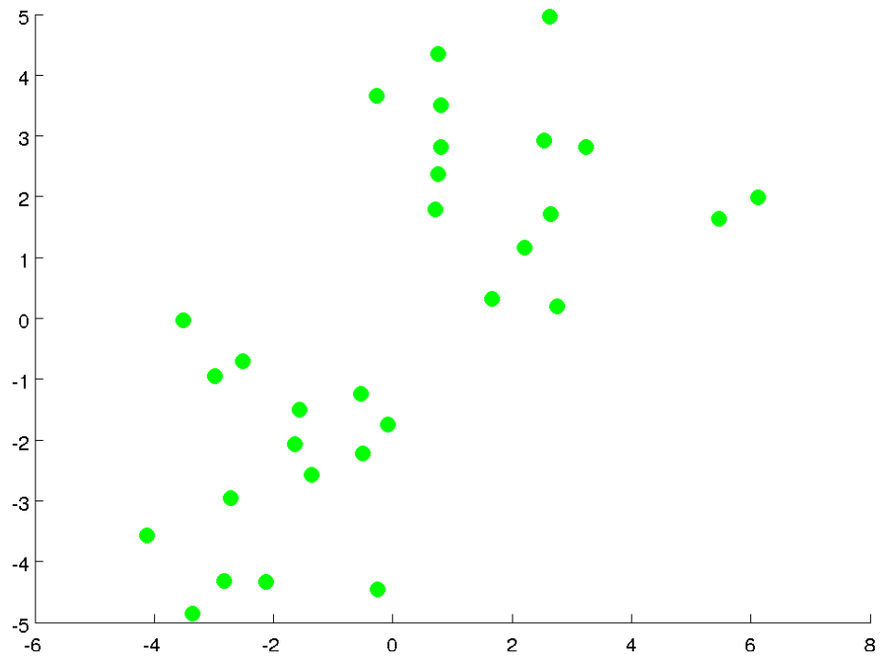
- **Goal:** Group ***similar*** data points together
- Desirable properties:
  - Points in same cluster → close
  - Points in different clusters → far
- *What defines similar?*



K-Means Overview

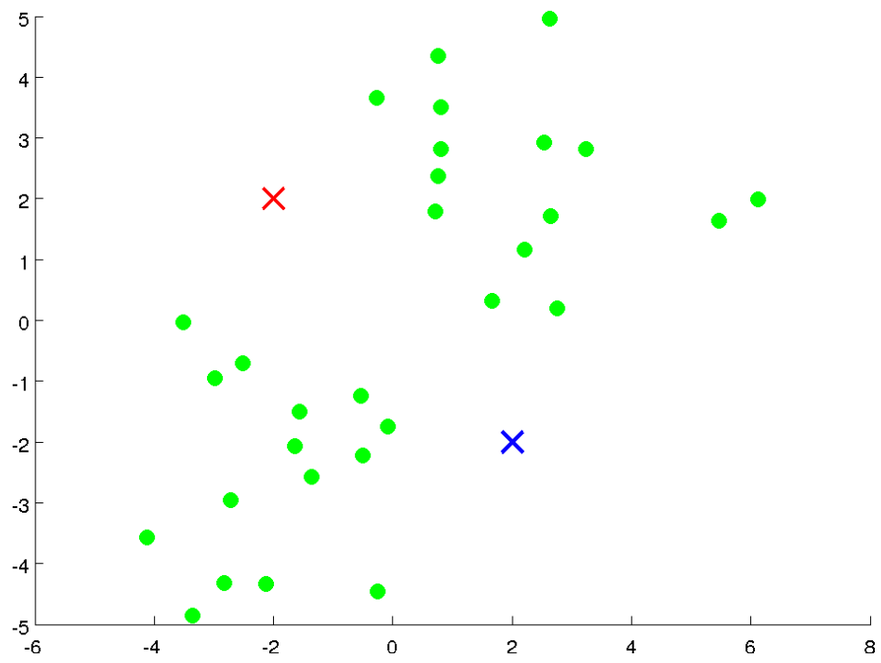
- Simplest clustering algorithm
- Surprisingly powerful
- Works for numerical vector data

- **Given:**
  - Data points $x_1, \ldots, x_n \in \mathbb{R}^d$
  - Number of clusters *K*
- **Goal:**
  - Partition into *K* clusters



K-Means

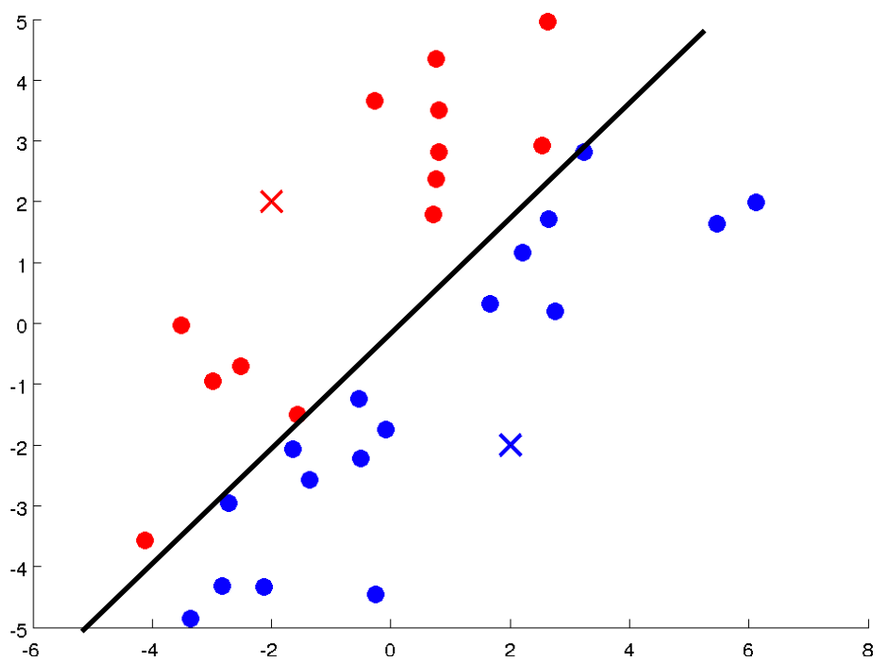
- Example: $K = 2$
  - Goal: separate data into 2 clusters
- Algorithm:
  - **Assign a center for each cluster**
  - Alternate between two steps:
    - Cluster Assignment Step: assign points to cluster whose center is closer
    - Centroid Update Step: compute mean of points in each cluster
  - Until we find 2 separate groups of points



K-Means

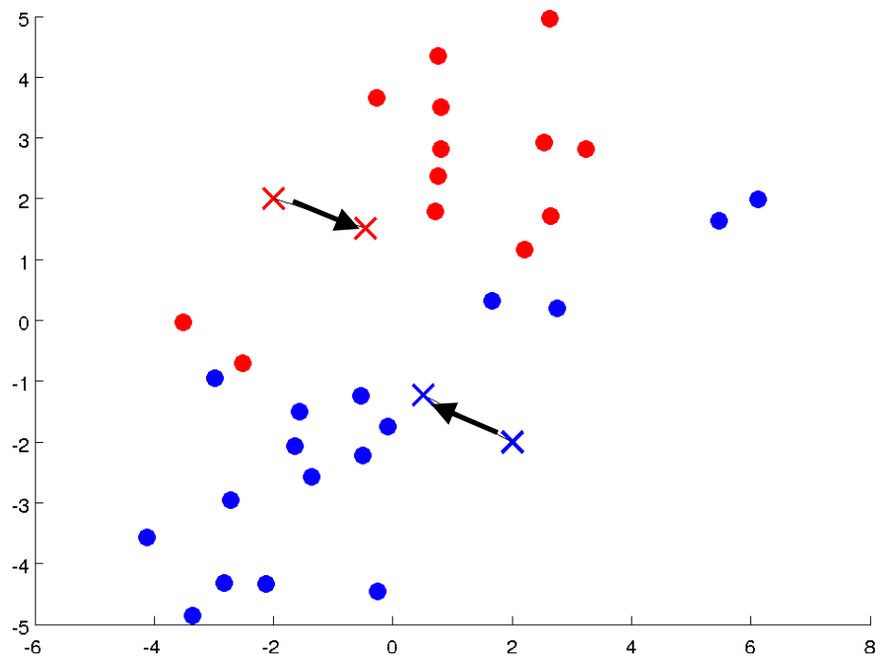
- Example: *K = 2*
  - Goal: separate data into 2 clusters
- Algorithm:
  - Assign a center for each cluster
  - Alternate between two steps:
    - **Cluster Assignment Step:** assign points to cluster whose center is closer
    - Centroid Update Step: compute mean of points in each cluster
  - Until we find 2 separate groups of points

Cluster assignment creates linear decision boundaries



K-Means

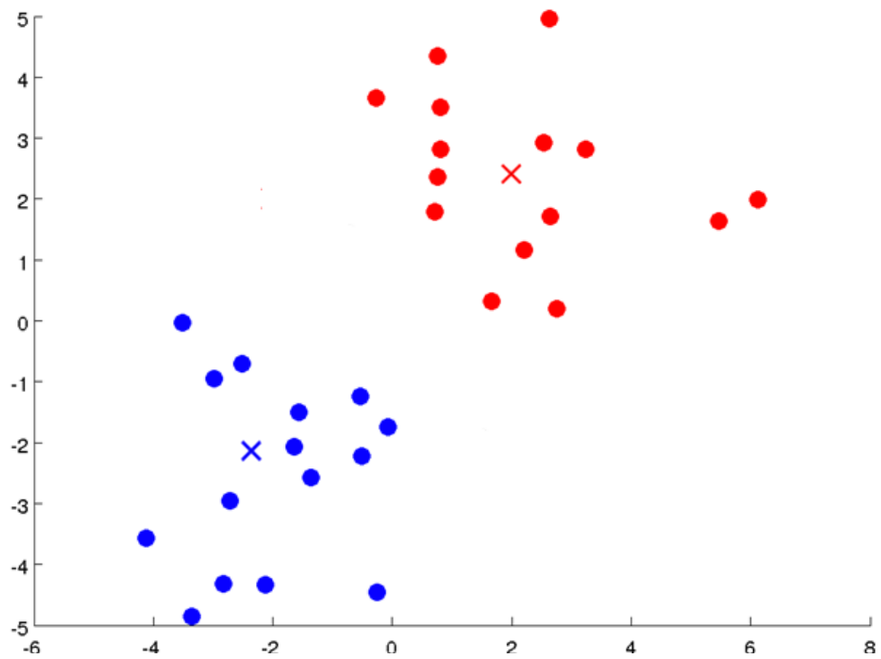
- Example: $K = 2$
  - Goal: separate data into 2 clusters
- Algorithm:
  - Assign a center for each cluster
  - Alternate between two steps:
    - Cluster Assignment Step: assign points to cluster whose center is closer
    - **Centroid Update Step:** compute mean of points in each cluster
  - Until we find 2 separate groups of points



K-Means

- Example: *K = 2*
  - Goal: separate data into 2 clusters
- Algorithm:
  - Assign a center for each cluster
  - Alternate between two steps:
    - Cluster Assignment Step: assign points to cluster whose center is closer
    - Centroid Update Step: compute mean of points in each cluster
  - **Until we find 2 separate groups of points**

Stop when:
- Centroids stop changing
- Or objective stops decreasing



K-Means Algorithm

- Randomly initialize K cluster centroids $\mu_1, \mu_2, \cdots, \mu_K \in \mathbb{R}^n$

Repeat{

for i = 1 to m

$c^{(i)}$:= index (from 1 to K) of cluster centroid closest to $x^{(i)}$

Cluster assignment step

for k = 1 to K

μ_k := average (mean) of points assigned to cluster k

Centroid update step

}

Optimization Objective

What is K-Keans minimizing?

- $c^{(i)}$ = Index of cluster (1, 2, … K) to which example $x^{(i)}$ is currently assigned
- μ_k = cluster centroid k ($\mu_k \in \mathbb{R}^n$)
- $\mu_{c^{(i)}}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned

Example:

$c^{(i)} = 5$

$\mu_{c^{(i)}} = \mu_5$

Optimization objective

$$J\left(c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K\right) = \frac{1}{m}\sum_{i=1}^{m} \left\| x^{(i)} - \mu_{c^{(i)}} \right\|^2$$

$$\min_{\substack{c^{(1)},\cdots,c^{(m)} \\ \mu_1,\cdots,\mu_K}} J\left(c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K\right)$$

Find cluster assignments & centroids to minimize **data distortion**: **distances (point, centroid)**

K-Means Algorithm

- Randomly initialize K cluster centroids $\mu_1, \mu_2, \cdots, \mu_K \in \mathbb{R}^n$

Repeat{

for i = 1 to m

$c^{(i)}$:= index (from 1 to K) of cluster centroid closest to $x^{(i)}$

for k = 1 to K

μ_k := average (mean) of points assigned to cluster k

}

Cluster assignment step

$$J\left(c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K\right) = \frac{1}{m}\sum_{i=1}^{m}\left\|x^{(i)} - \mu_{c^{(i)}}\right\|^2$$

Centroid update step

$$J\left(c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K\right) = \frac{1}{m}\sum_{i=1}^{m}\left\|x^{(i)} - \mu_{c^{(i)}}\right\|^2$$

repeat until convergence

Objective-based Clustering

- Input:
  - K (number of clusters)
  - Training set $\{x^{(1)}, x^{(2)}, x^{(3)}, \cdots, x^{(m)}\}$
  - Distance/dissimilarity measure $\mathrm{d}(x^{(i)}, x^{(j)})$

- Goal: output a partition of the data
  - K-means: Euclidean distance
  - K-median: Absolute distance
  - K-center: minimize the maximum radius

Summary

- K-means definition & how it works
- K-means algorithm
  - 2 update steps
  - Optimization objective
  - General objective-based clustering
  - **Initialization**

Initialization

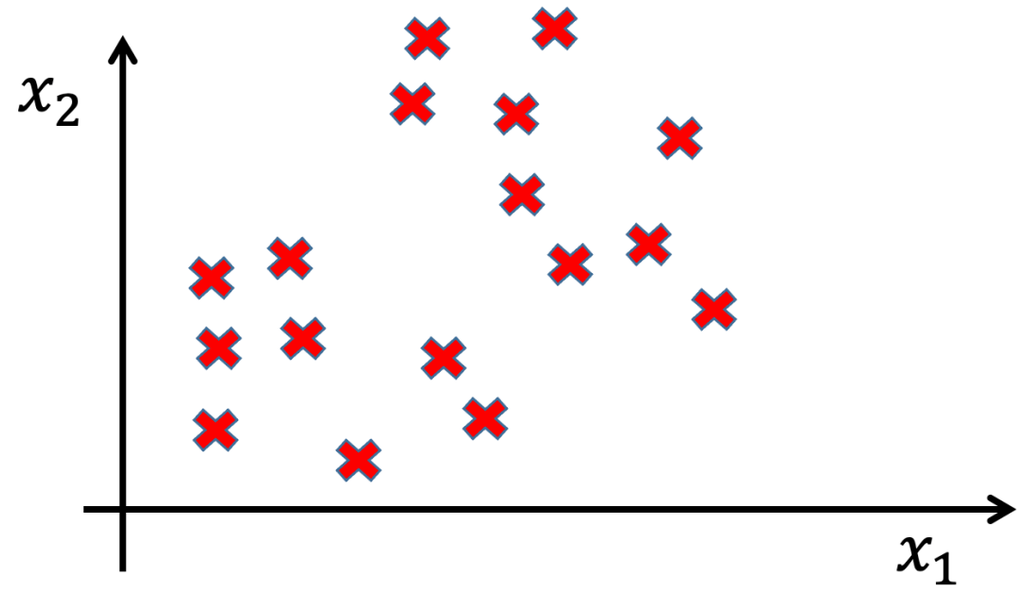
- Initialization
  - Choose the first set of cluster centroids
  - Choose the number of clusters
- Why Initialization is Important
  - Bad initialization → bad clustering
- Same data
  - Two different initializations
  - Two very different final clusterings

Initialization of Centroids

- **Random Initialization**
- Multiple Random Restarts
- Furthest Point Heuristic
- K-means++ (very important)

Random Initialization

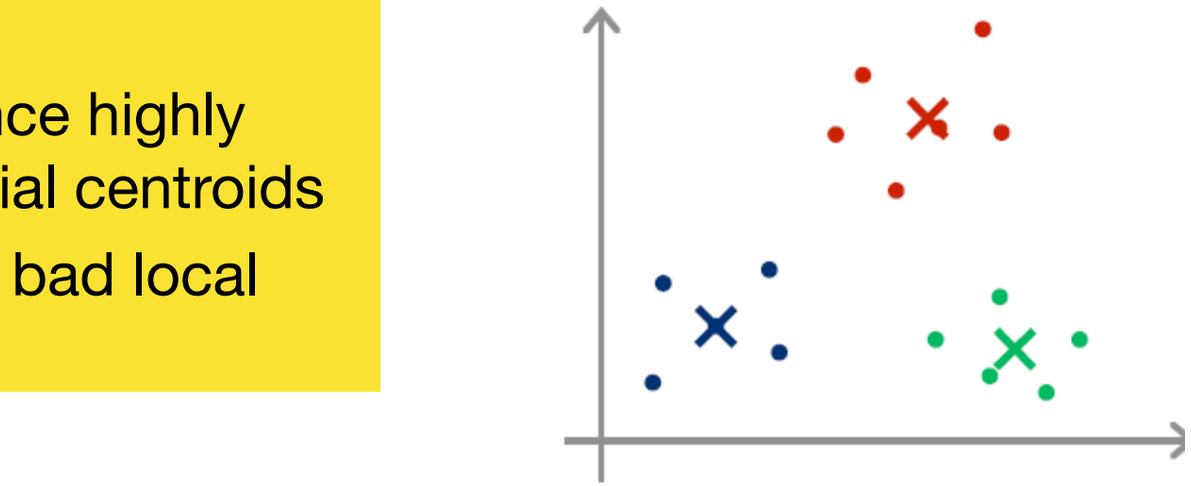
- Randomly pick K training examples
- Set $\mu_1, \mu_2, \cdots, \mu_K$ equal to those K examples

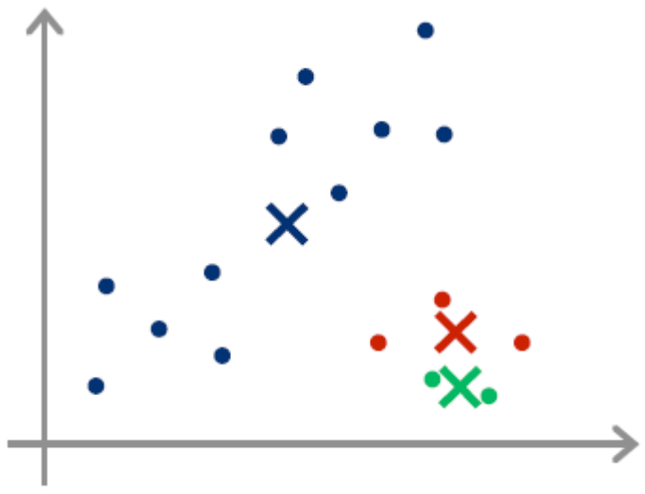


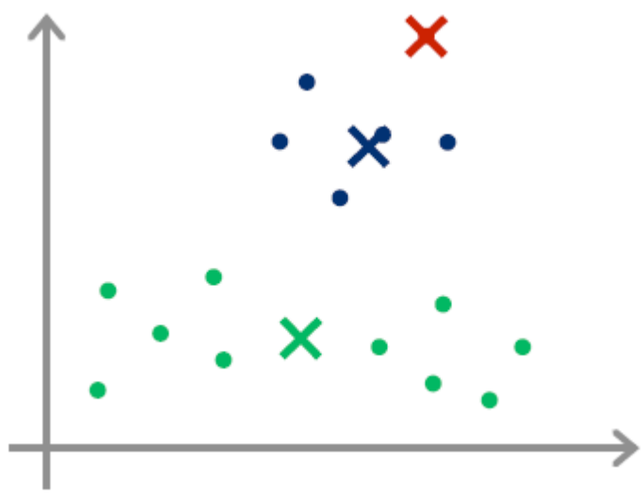
- + + Simple
- ++ Good convergence
- - - Can get stuck in bad local minima (highly sensitive)

Random Initialization

highly sensitive:
- good convergence highly dependent on initial centroids
- can get stuck in bad local minima

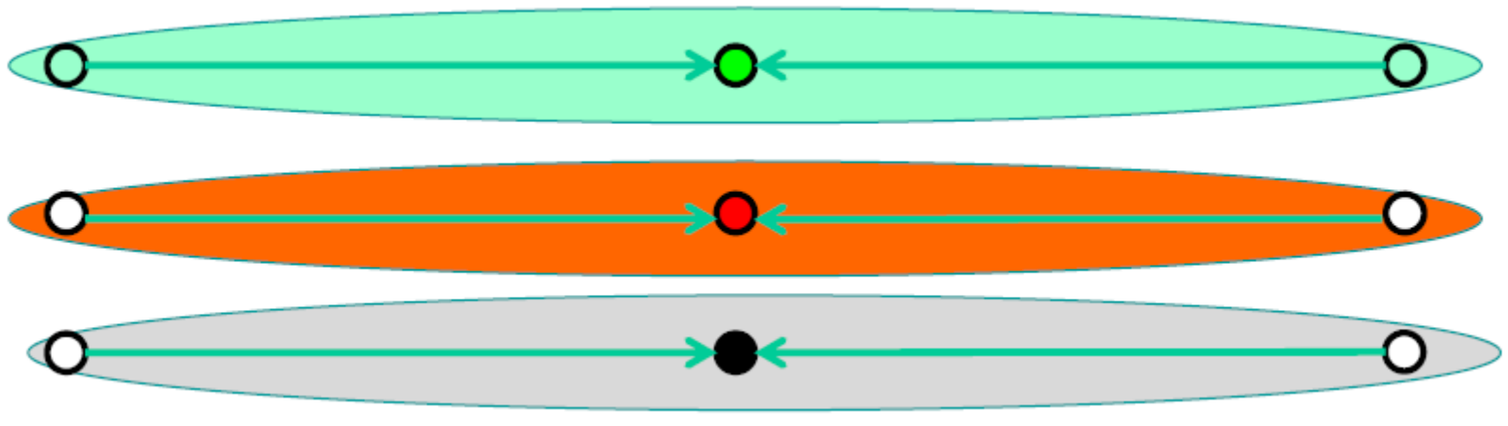




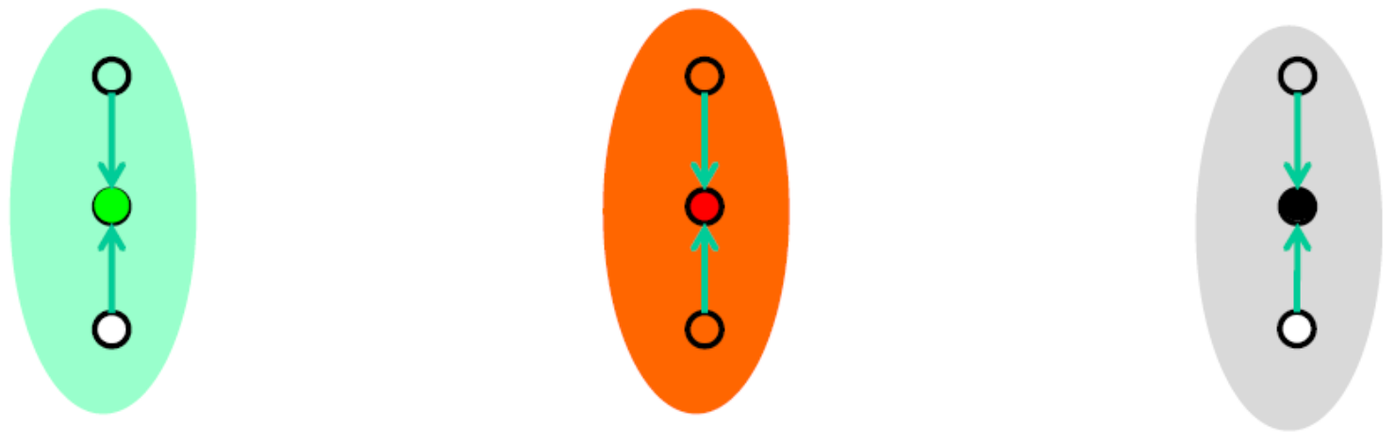


Random Initialization

- - algorithm can get stuck in bad local minimum



Ideal clustering:
Close points within clusters
Far points between clusters



Initialization of Centroids

- Random Initialization
- **Multiple Random Restarts**
- Furthest Point Heuristic
- K-means++

Multiple Random Restarts

- Run algorithm many times
- Choose best objective value
- Tradeoff:
  - More computation
  - Better reliability

Multiple Random Restarts

For i = 1 to 100 {

Randomly initialize K-means

Run K-means. Get $c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K$

Compute the cost function (distortion)

$J(c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K)$

}

Pick clustering that gave the lowest cost $J(c^{(1)}, \cdots, c^{(m)}, \mu_1, \cdots, \mu_K)$

Tradeoff:
- - Inefficient (more computation)
+ + More reliable

Initialization of Centroids

- Random Initialization
- Multiple Random Restarts
- **Furthest Point Heuristic**
- K-means++

Furthest Point Heuristic

- **Idea:**
    - Choose centroids far apart
- **Algorithm:**
    - Pick first randomly
        - Pick next farthest point
        - Repeat
- **Intuition:**
    - Encourages spread

Furthest Point Heuristic

Choose μ_1 arbitrarily (or at random)

For j = 2 to K

Pick μ_j among data points $x^{(1)}, x^{(2)}, \cdots, x^{(m)}$ that is farthest from previously chosen $\mu_1, \mu_2, \cdots, \mu_{j-1}$

select cluster centroids that are as far as possible from each other

+ + encourages spread

Initialization of Centroids

- Random Initialization
- Multiple Random Initialization
- Furthest Point Heuristic
- **K-means++ (very important)**

K-Means++

- Probability proportional to squared distance: $P(x) \propto D(x)^2$
- Where:
  - D(x) = distance to nearest chosen centroid
- Benefits:
  - Theoretical guarantees
  - Often works in one run
  - Default in practice

K-Means++

- Interpolate between random and furthest point initialization
- Idea: Let $\mathrm{D}(x)$ be the distance between a point x and its nearest center. Chose the next center proportional to $\mathrm{D}^2(x)$.

Choose μ_1 arbitrarily (or at random)

For j = 2 to K

Pick μ_j among data points $x^{(1)}, x^{(2)}, \cdots, x^{(m)}$ according to the distribution

$$\Pr(\mu_j = x^{(i)}) \propto \min_{j'<j} \|x^{(i)} - \mu_{j'}\|^2$$

proportional to distance squared

combine benefits of random sampling with those of furthest point

K-Means++

- Interpolate between random and furthest point initialization
- Idea: Let $\mathrm{D}(x)$ be the distance between a point x and its nearest center. Chose the next center proportional to $\mathrm{D}^{\alpha}(x)$.

- $\alpha = 0$: random sampling
- $\alpha = \infty$: furthest point
- $\alpha = 2$: K-means++

scikit-learn uses K-means++ as default

a single run may work well

Choice of K

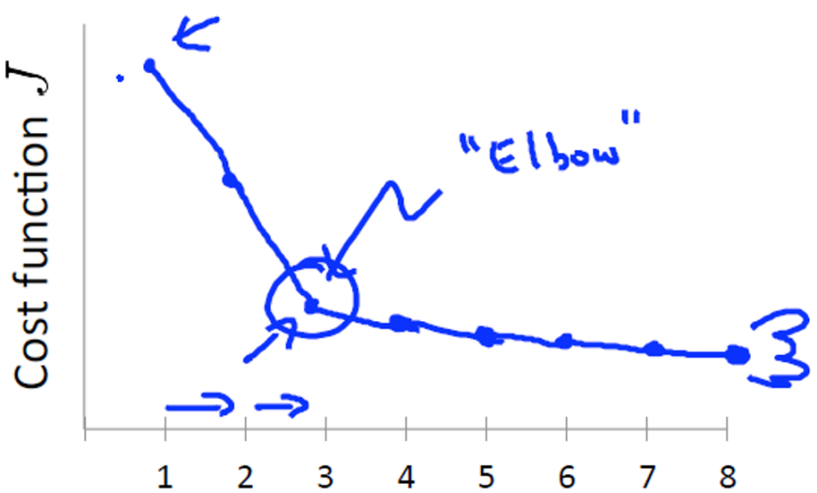
Choice of K

- Given a set of data points, how do we specify K?
- No universal answer

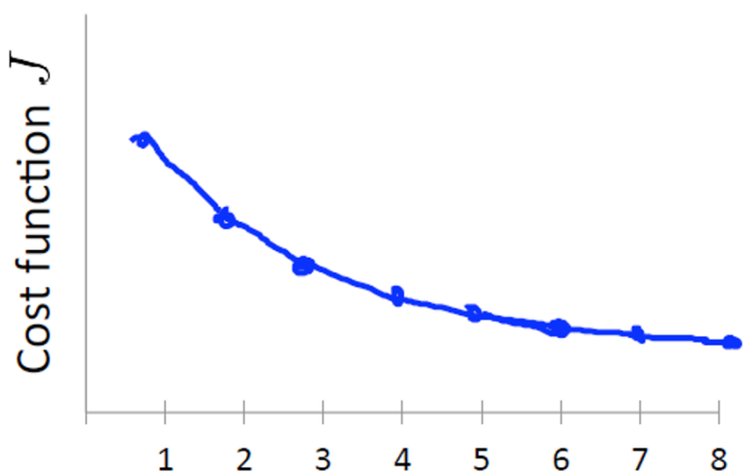
Choice of K

- Methods for choosing K
- Elbow method
- Downstream Task Performance

Elbow method



- **Algorithm:**
  - Run K-means for multiple K
  - Plot objective/cost J(K) for each k
  - Look for:
    - Sharp drop → then flattening



- Sometimes no clear elbow

Downstream Task Performance

- If we use clusters as input to another task:
    - Classification, Anomaly detection, or Compression
- For a given task that uses clusters as input:
    - Try multiple k
    - Pick k that gives best task performance
- Example: clustering is used for anomaly detection,
  → choose K that gives best detection performance

When Does K-Means Work Well?

✔ Spherical clusters

✔ Similar cluster sizes

✔ Euclidean geometry meaningful

When Does It Fail?

✘ Non-convex clusters

✘ Different densities

✘ Categorical data

✘ Outliers

✘ Spherical clusters

Summary

- K-means:
  - Cluster assignment
  - Centroid update
- Minimizes squared distance objective
- Converges to local minimum
- Initialization is critical
- K-means++ is preferred
- Choosing K is problem-dependent